

Reviewer Pack — Minimal Local Index vs Communication Complexity Rank Variance...

Entry f2bf4d9d679f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 10:46:03 UTC

Minimal Local Index vs Communication Complexity Rank Variance Ratio

Entry ID: f2bf4d9d679f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 10:46:03 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Local Systems) **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

For every n -communication complexity instance φ , the ratio of the minimal local index $I_{\min}(\varphi)$ to the communication complexity rank variance σ_{φ} is bounded by a constant c , i.e., $|I_{\min}(\varphi)/\sigma_{\varphi}| \leq c$.

Rationale (proposer's reasoning):

Local systems in algebraic topology have been used to study the structure of complex spaces. The minimal local index measures the complexity of such structures. Communication complexity rank variance captures the complexity of communication tasks in distributed computing. The conjecture suggests a potential connection between these two concepts, which could provide insights into the nature of distributed computations.

Taxonomy category: ALGEBRAIC_TOPOLOGY_X_COMMUNICATION_COMPLEXITY_RANK_VARIANCE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 07a98d396aaef7b3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every n -communication complexity instance φ , the ratio of the minimal local index $I_{\min}(\varphi)$ to the communication complexity rank variance σ_{φ} must be within $\pm 5\%$ of a constant c .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local index" AND "communication complexity rank variance" - "Algebraic Topology" AND "Communication Complexity" AND ("matrix rank variance" OR "local systems") - "I_min" AND " σ_ϕ " AND ("communication complexity" OR "algebraic topology")

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = A[i][i]
            for j in range(i, n + 1):
                A[i][j] /= factor
            for j in range(n):
                if j != i:
                    factor = A[j][i]
                    for k in range(i, n + 1):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        n = len(A)
        C = [[0 for _ in range(n)] for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C
```

```

def communication_complexity_rank_variance(A):
    n = len(A)
    rank = 0
    U, S, Vt = gaussian_elimination(A)
    for s in S:
        if abs(s) > 1e-10:
            rank += 1
    variance = sum((s**2 for s in S)) / rank
    return variance

def minimal_local_index(A):
    n = len(A)
    index = 0
    for i in range(n):
        for j in range(i+1, n):
            if A[i][j] != 0:
                index += 1
    return index

instances_tested = 30
n_max = 40
total_ratio = 0.0

for _ in range(instances_tested):
    n = random.randint(5, n_max)
    A = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]
    I_min = minimal_local_index(A)
    sigma_phi = communication_complexity_rank_variance(A)

    if sigma_phi == 0:
        continue

    ratio = abs(I_min / sigma_phi)
    total_ratio += ratio

mean_ratio = total_ratio / instances_tested
conjecture_holds = all(3.95 <= ratio <= 4.05 for ratio in [mean_ratio])

return {
    "metric_name": "Ratio of Minimal Local Index to Communication Complexity Rank Variance",
    "metric_value": mean_ratio,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"mean_ratio={mean_ratio}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_ratio = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_179cec7b.py", line 101, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_179cec7b.py", line 75, in run_trial
    sigma_phi = communication_complexity_rank_variance(A)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_179cec7b.py", line 51, in communication_complexity_rank_variance
    U, S, Vt = gaussian_elimination(A)
                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_179cec7b.py", line 31, in gaussian_elimination
    A[i][j] /= factor
    ~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's claim. | next: Investigate and fix the crash in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14452
2	propose	ollama_remote	glm4:latest	0	0	12478
3	preregistration	ollama_remote	glm4:latest	0	0	13901
4	novelty	ollama_remote	glm4:latest	0	0	8281
5	novelty	ollama_remote	glm4:latest	0	0	8983
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16571
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11951
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10355
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11227
10	judge	ollama_remote	glm4:latest	0	0	16189

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 124386 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/f2bf4d9d679f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f2bf4d9d679f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f2bf4d9d679f.tar.gz` (if generated)